CSCI 3060U Phase 2: AI Use Acknowledgement

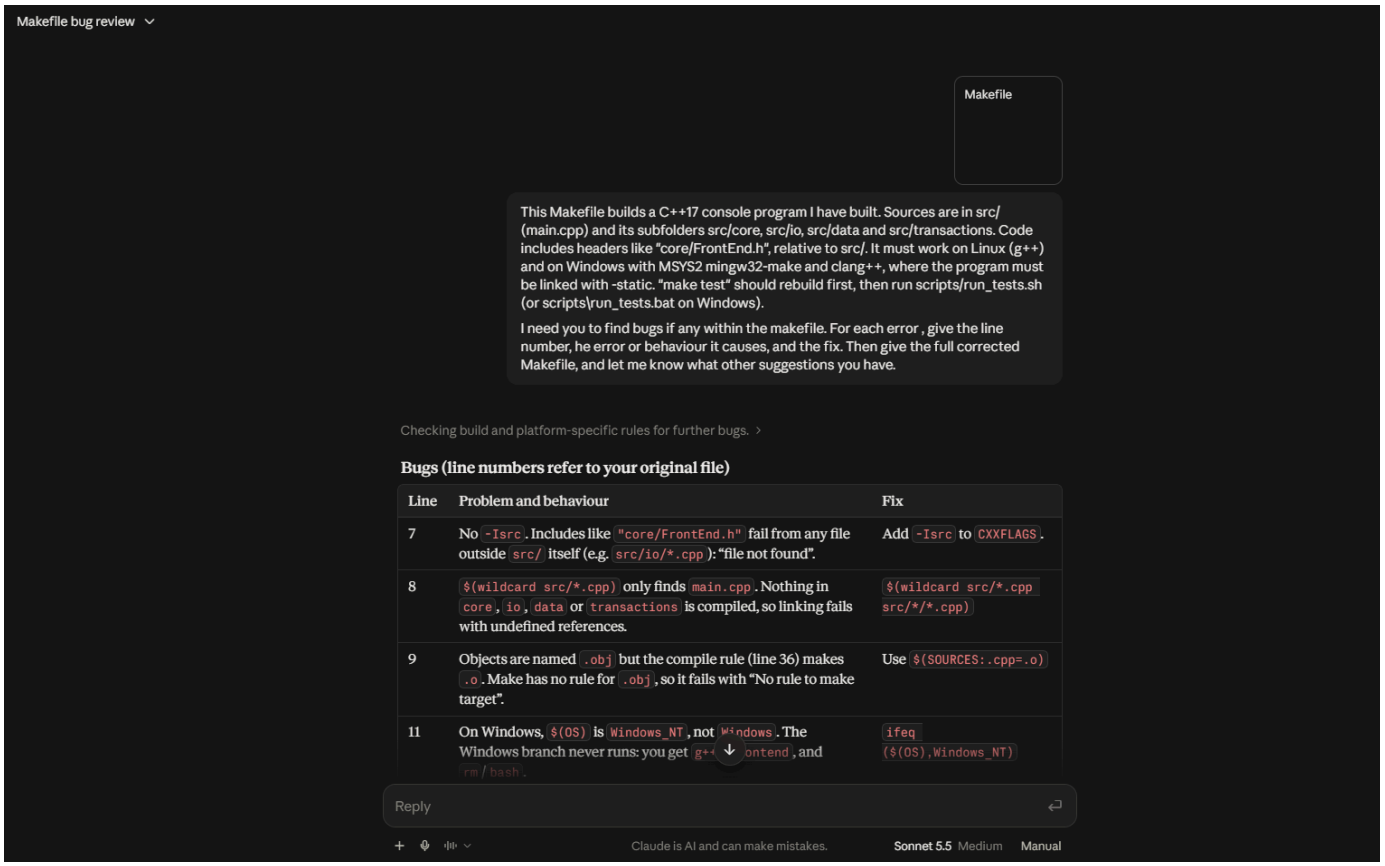
Team Members: Evan Harder, Muhammad Niazi, Maheen Zahir

Generative AI was used in this project phase. As required by the Phase 2 handout, the complete responses for AI conversation used on this phase is attached after this page, unedited and in full.

What the AI was used for:

- Conversation 1: Asked to review makefile and suggest any improvements that should be made all throughout the repo.

Conversation 1 image(s):



Makefile bug review

14	`LDFLAGS` is empty on Windows, so the program is not statically linked.	`LDFLAGS = -static`
33	`$<` is only the first object file. Linking fails with undefined references once more than one object exists.	Use `$^`.
37	Missing `-c`. The compiler tries to link each `.cpp` into an executable, which fails with "undefined reference to main" for every file except `main.cpp`, and never produces a proper `.o`.	Add `-c`.
40	`test` has no prerequisites, so it does not rebuild first.	`test: all`
44	The recipe is indented with spaces, not a tab: "missing separator".	Indent with a real tab.
16, 23	The test scripts hardcode `frontend.exe` and `./frontend` instead of using `$(TARGET)`. They can drift out of sync.	Use `$(TARGET)`.

Lines 46 (`-include` of `.d` files) and 37's `-MMD -MP` are fine, and they only start working once the `.obj` mismatch is fixed.

Corrected Makefile

Recipe lines must start with a tab character.

makefile

```
# Makefile for the Front End (CSCI 3060U course project)
#
#   make              build the Front End (frontend, or frontend.exe on Windows)
#   make test         build, then run every test in TestCases/
#   make clean        delete the compiled program and object files

CXXFLAGS = -std=c++17 -Wall -Isrc
SOURCES = $(wildcard src/*.cpp src/*/*.cpp)
OBJECTS = $(SOURCES:.cpp=.o)

ifeq ($(OS),Windows_NT)
    CXX = clang++
    TARGET = frontend.exe
    LDFLAGS = -static
```

Reply

Claude is AI and can make mistakes. Sonnet 5.5 Medium Manual

Makefile bug review

```
#   make clean        delete the compiled program and object files

CXXFLAGS = -std=c++17 -Wall -Isrc
SOURCES = $(wildcard src/*.cpp src/*/*.cpp)
OBJECTS = $(SOURCES:.cpp=.o)

ifeq ($(OS),Windows_NT)
    CXX = clang++
    TARGET = frontend.exe
    LDFLAGS = -static
    RM = cmd /c del /q
    RUN_TESTS = cmd /c scripts\run_tests.bat $(TARGET)
    FIX_PATH = $(subst /,\,$(1))
else
    CXX = g++
    TARGET = frontend
    LDFLAGS =
    RM = rm -f
    RUN_TESTS = bash scripts/run_tests.sh ./$(TARGET)
    FIX_PATH = $(1)
endif

.PHONY: all test clean

all: $(TARGET)

# Links every object file into the Front End program.
$(TARGET): $(OBJECTS)
    $(CXX) $(LDFLAGS) -o $@ $^

# Compiles one .cpp file; -MMD -MP also records which headers it uses.
%.o: %.cpp
    $(CXX) $(CXXFLAGS) -MMD -MP -c $< -o $@

# Rebuilds if needed, then runs the full requirements test suite.
test: all
    $(RUN_TESTS)
```

Reply

Claude is AI and can make mistakes. Sonnet 5.5 Medium Manual

Makefile bug review

```
%.o: %.cpp
    $(CXX) $(CXXFLAGS) -MMD -MP -c $< -o $@

# Rebuilds if needed, then runs the full requirements test suite.
test: all
    $(RUN_TESTS)

clean:
    -$(RM) $(call FIX_PATH,$(TARGET) $(OBJECTS) $(OBJECTS:.o=.d))

-include $(OBJECTS:.o=.d)
```

Other suggestions

- **MSYS2 shell quirk:** if you run `mingw32-make` from an MSYS2 shell, `sh.exe` is on the PATH. Then `cmd /c` can be mangled into a path (`/c` becomes `C:\`), and backslashes in `RUN_TESTS` get eaten. Either run from `cmd` /PowerShell, or detect the shell and use `rm -f` and `bash scripts/run_tests.sh` there. Writing `cmd //c` also works in MSYS2 but breaks in plain cmd.
- **Keep clean from failing the build:** the `-` prefix already ignores errors from missing files, which is correct. Keep it.
- **Build directory:** objects and `.d` files currently sit next to the sources. An `obj/` directory would keep `src/` clean, but it needs `mkdir` handling that differs on Windows, so only do it if it matters.
- **Warnings:** consider adding `-Wextra`, and `-g` for debugging (or `-O2` for release).
- **Wildcard depth:** `src/*/*.cpp` covers one subfolder level. If you ever nest deeper, add `src/*/*/*.cpp`.
- **Line endings:** if `run_tests.sh` has CRLF endings from Windows, bash on Linux will fail with `\r` errors. Add a `.gitattributes` line: `*.sh text eol=lf`.
- **Declared dependency:** `test` depends on `all`, so `make test` won't run if the build fails. That is the behaviour you want.

Reply

Claude is AI and can make mistakes. Sonnet 5.5 Medium Manual